Introduction

If you've written Python for more than a few weeks, you've probably used a decorator without fully knowing what it does. `@app.route`, `@staticmethod`, `@pytest.fixture`, `@lru_cache` — they're everywhere, and most tutorials treat them like syntax you memorize rather than a mechanism you understand.

That gap matters more than it looks like it should. Interviewers ask about decorators not because you'll write one every day, but because explaining one forces you to demonstrate that you understand functions as values, scope, and closures — three ideas that show up constantly in real Python code, from callbacks to `sorted()` to Flask routes. And there's a genuinely nasty bug hiding inside almost every decorator you'll ever write: it silently deletes the identity of the function it wraps. Miss that, and you'll spend an afternoon debugging something that looks like a completely unrelated problem — a broken introspection tool, a missing docstring, a test framework that can't find your test.

This lesson builds the whole picture from the ground up: functions as first-class values, closures, the decorator pattern itself, and the fix for the identity bug most people don't know exists until it bites them.

Problem Overview

There's no LeetCode-style input/output here — this is a conceptual problem, and it's this: **how do you add behavior to a function — timing, logging, caching, access control — without modifying the function's own code, and without losing information about what that function actually is?**

Rewritten as a sequence of sub-problems:

1. Can you treat a function as data — store it, pass it around, return it from another function?
2. If an inner function uses a variable from an outer function, and the outer function has already finished running, how does the inner function still have access to that variable?
3. Given (1) and (2), how do you build a wrapper that adds behavior around any function, generically?
4. Once you build that wrapper, what breaks about the original function, and how do you fix it?

Each one builds on the last. That's exactly how decorators are usually taught badly — as syntax — and exactly why they should be taught as a sequence of small, provable facts.

Example

Start with the simplest possible proof that functions are values:

```
def greet():
    return "hello"

say_hi = greet
print(say_hi())  # "hello"
```

Nothing about `greet` changed. `say_hi` isn't a copy of the *result* of `greet` — it's a reference to the same function object. Both names point at the same thing in memory.

Next, the closure example:

```
def make_counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment

counter = make_counter()
print(counter())  # 1
print(counter())  # 2
```

`make_counter()` runs, returns `increment`, and finishes. By every normal rule of scope, `count` should be gone. It isn't — calling `counter()` again shows it's still there, still climbing. That's the whole mystery this lesson is built around.

Intuition

Here's how an engineer arrives at this without being told the answer in advance.

Start from something you already accept: `sorted(names, key=len)`. You're passing the function `len` — not calling it, not stringifying it — as a plain argument. If a function can be an argument, it can be a return value too. Nothing in the language distinguishes functions from any other object; that symmetry is what makes the rest of this work.

Now ask: if `make_counter` returns `increment`, and `increment` references `count`, what does Python do with `count` when `make_counter` exits? The naive answer — "it's deleted, like any local variable" — would make `increment` crash the moment you called it. But it doesn't crash. So Python

must be doing something smarter: it looks at `increment`'s body, notices it references a variable from the enclosing scope, and keeps that variable alive specifically because `increment` still needs it. That bundle — the function plus the variables it captured — is a **closure**.

Once closures click, decorators aren't a new concept, they're an application of one. If you can write a function that takes another function and returns a *new* function that wraps it, you've built a decorator by hand. The `@` syntax is just sugar for calling that wrapping function and reassigning the result to the original name:

```
@my_decorator
def do_work():
    ...

# is exactly equivalent to:

def do_work():
    ...
do_work = my_decorator(do_work)
```

That's the entire trick. No new execution model, no magic — just a function call and a reassignment.

Brute Force Solution

"Brute force" here means: add the behavior (say, timing) by hand, inside every function you want timed.

```
import time

def slow_task():
    start = time.perf_counter()
    # actual work
    total = sum(i * i for i in range(1_000_000))
    elapsed = time.perf_counter() - start
    print(f"slow_task took {elapsed:.4f}s")
    return total
```

Advantages: obvious, no indirection, easy to step through in a debugger.

Disadvantages: you repeat the timing boilerplate in every function that needs it. Change how you log timing — say, switch from `print` to a logger — and you have to edit every single function. This is the exact repetition that should make you reach for a decorator.

Complexity: O(1) extra time and space overhead per call beyond the function's own cost — the inefficiency here isn't computational, it's maintenance cost.

Optimal Solution

Write the timing logic once, as a decorator that wraps *any* function:

```
do_work()
   │
   ▼
wrapper()            ← this is what actually gets called now
   ├─ start timer
   ├─ call original do_work()
   ├─ stop timer, print elapsed
   └─ return do_work()'s result
```

The decorator function takes `func` as a parameter, defines `wrapper` as a closure around it (exactly like `increment` closed over `count`), and returns `wrapper`. When you call the decorated function, you're actually calling `wrapper`, which calls the original function internally.

Python Code

```
import functools
import time


def timer(func):
    """Decorator that prints how long `func` took to run."""

    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} took {elapsed:.4f}s")
        return result

    return wrapper


@timer
def slow_task(n: int) -> int:
    """Sum of squares from 0 to n-1."""
    return sum(i * i for i in range(n))


if __name__ == "__main__":
    slow_task(1_000_000)
```

```
print(slow_task.__name__)   # "slow_task", not "wrapper"
print(slow_task.__doc__)    # original docstring preserved
```

Code Walkthrough

- `def timer(func):` — the decorator itself; it accepts the function being decorated.
- `def wrapper(*args, **kwargs):` — the closure. `*args, **kwargs` let it wrap *any* function signature, not just no-argument ones.
- `func(*args, **kwargs)` — calls the original function with whatever arguments were passed to `wrapper`, and captures the result.
- `func.__name__` inside the print statement — note this reads the *original* function's name, not the wrapper's, because `functools.wraps` hasn't affected `func`, only `wrapper`'s metadata.
- `return wrapper` — the decorator hands back the new function; this is what `slow_task` gets reassigned to.
- `@functools.wraps(func)` — copies `__name__`, `__doc__`, `__module__`, and a few other attributes from `func` onto `wrapper`, so that after decoration, `wrapper` *looks* like `func` from the outside.

Dry Run

1. `slow_task = timer(slow_task)` runs at import time (triggered by `@timer`).
2. `timer` receives the original `slow_task` as `func`, defines `wrapper`, applies `functools.wraps(func)` to `wrapper`, and returns `wrapper`.
3. The name `slow_task` in the module now points to `wrapper` — but `wrapper.__name__` was copied from the original, so it reports `"slow_task"`.
4. Calling `slow_task(1_000_000)` actually calls `wrapper(1_000_000)`.
5. `wrapper` records `start`, calls `func(1_000_000)` (the real computation), records `elapsed`, prints the timing line, and returns the sum.
6. `slow_task.__name__` prints `"slow_task"` and `slow_task.__doc__` prints the original docstring — because of `functools.wraps`. Remove that line and both would read `"wrapper"` / `None`.

Complexity Analysis

- **Time:** O(1) overhead added by the wrapper itself (two timer reads, one print), on top of whatever `func` costs. The wrapping doesn't change the asymptotic complexity of the wrapped

function.

- **Space:** O(1) extra — the closure holds one reference (`func`) plus whatever `functools.wraps` copies over (references to strings/dicts, not deep copies).
- These are correct because the decorator never touches the algorithm inside `func` ; it only adds bookkeeping around the call.

Alternative Solutions

Class-based decorators, using `__call__` , are a reasonable alternative when the decorator needs to hold more complex state (e.g., counting calls across many functions):

```
class Timer:
    def __init__(self, func):
        functools.update_wrapper(self, func)
        self.func = func

    def __call__(self, *args, **kwargs):
        start = time.perf_counter()
        result = self.func(*args, **kwargs)
        print(f"{self.func.__name__} took {time.perf_counter() - start:.4f}s")
        return result
```

Functionally equivalent for this case, but heavier. Stick with the closure-based version unless you need per-decorator state that doesn't cleanly live in a closure variable.

Edge Cases

- **Function with no arguments** — handled by `*args, **kwargs` defaulting to empty.
- **Function that raises an exception** — as written, `wrapper` doesn't catch it, so the exception propagates normally and the timing print never happens. Wrap `func(*args, **kwargs)` in `try/finally` if you need timing even on failure.
- **Decorating a method** — `self` arrives as the first positional argument in `*args` , so this works unchanged.
- **Decorating an already-decorated function** — stacking decorators works because each one just wraps the previous wrapper; order matters (`@a` above `@b` means `a(b(func))`).
- **Missing `functools.wraps`** — not a crash, but a silent correctness bug: `__name__` , `__doc__` , and `__module__` all report the wrapper's values instead of the original's.

Common Mistakes

1. **Forgetting `functools.wraps`**, which breaks introspection, documentation generators, and any code that keys off `__name__` (some test runners and CLI frameworks do).
2. **Not accepting `*args, **kwargs`** in the wrapper, which breaks the decorator for any function that isn't zero-argument.
3. **Forgetting to `return result`** from the wrapper, silently turning every decorated function into one that returns `None`.
4. **Confusing `@decorator` with `@decorator()`** — a decorator factory (one that takes its own arguments, like `@retry(times=3)`) needs an extra layer of nesting; forgetting it causes a `TypeError` at decoration time.
5. **Mutating shared state in a closure without `nonlocal`**, which raises `UnboundLocalError` on assignment inside the inner function.

Interview Questions

- What's the difference between a closure and a regular nested function?
- Why does `functools.wraps` matter, and what specifically does it copy?
- How would you write a decorator that also accepts arguments (like `@retry(times=3)`)?
- Can you write a decorator that works on both regular functions and methods?
- What happens if you stack multiple decorators on one function — what order do they execute in?

Similar Problems

- **LeetCode-style "Design a Rate Limiter"** — often solved with a decorator-and-closure pattern that tracks call timestamps.
- **`functools.lru_cache` implementation** — a real-world decorator that uses closures to store a cache dict per decorated function.
- **Memoization exercises** — writing your own cache decorator is a natural next step after this lesson.
- **Logging decorator exercises** — same shape as the timer here, swapping the timing logic for structured logging.

Key Takeaways

- Functions in Python are ordinary values — they can be assigned, passed, and returned like anything else.

- A closure is a function bundled with the variables it captured from its enclosing scope, kept alive specifically because the inner function still needs them.
- A decorator is just a function that takes a function, defines a closure around it, and returns that closure — the `@` syntax is sugar for a call-and-reassign.
- Wrapping a function this way silently replaces its `__name__` and `__doc__` with the wrapper's own — `functools.wraps` is the one-line fix, and it should be in every decorator you write.

Watch the Video

This lesson is easier to absorb watching it run live — see the closure survive scope exit, watch the wrapped function lose its identity, and watch `functools.wraps` restore it in real time: {LINK}

About the Series

This is part of **Fun with Learning Technology**, a series built around one idea: don't just run the code, figure out why it works. Instead of memorizing syntax, each lesson takes one Python concept apart until the "magic" disappears and what's left is a small, provable mechanism you can reason about on your own.

Call To Action

If this helped the closure-and-decorator picture finally click, do three things: like the video on YouTube so more developers find it, subscribe to the channel for the rest of the Python series, and subscribe to the Substack for deeper written breakdowns like this one. Drop a comment with your guess from the video, or with a decorator pattern you'd like explained next — every lesson in this series comes from a question someone actually asked.

Quick Review

What's the pattern: a function is used as a value, passed/returned/stored?

First-class functions — treat functions like any other object (assign, pass, return).

What lets an inner function remember variables after the outer function returns?

A closure — it keeps a reference to the enclosing scope's variables, not a copy.

Why doesn't Python garbage-collect the outer function's variables?

The inner function holds a reference (cell) to them, so refcount stays above zero.

What problem do decorators solve?

Adding behavior (logging, timing, auth) around a function without editing its body.

What does `@my_decorator` above a function actually do?

Sugar for `func = my_decorator(func)` — rebinds the name to the wrapper's return value.

Why wrap decorators with `functools.wraps`?

Without it, `wrapper` overwrites the original's `__name__`/`__doc__`, breaking introspection and debugging.
